Research Library with Airflow

INFO-H420 — Assignment 3
Group members: TO BE COMPLETED
GitHub repository: https://github.com/MustafaTarek77/research-library

DRAFT: This report describes verified local execution. Google Drive transfers have been added through rclone. The complete updated DAG and real Discord delivery still need final verification. Add the group members and update this report against the latest run evidence before submission.

Approach and architecture

We implemented one scheduled Apache Airflow 3.3.2 DAG running through `airflow standalone` in WSL2. Every five minutes it scans a configured papers directory, identifies filenames not already stored, converts PDFs to Markdown with PyMuPDF4LLM, and sends the complete extracted text to a local Ollama model. The model returns a title, summary, keywords, and category as schema-constrained JSON. Python parses the response with json.loads, checks required field types, and retries invalid output once.

SQLite stores one row per unique filename. A self-contained HTML page is regenerated from committed records, with text search, category filtering, and relative links to the PDFs. The notification task posts each pending paper to a configured Discord webhook. The intended synced layout is ResearchLibrary/papers and ResearchLibrary/output/index.html. The application's database, intermediate files, logs, and Airflow state remain outside that directory. An optional rclone task downloads PDFs before discovery, and another uploads the generated page before notification. Desktop-synced folders use the same pipeline with those transfers disabled.

Reliability and observability

Discovery, conversion, extraction, storage, publication, and notification are visible tasks. Processing tasks are mapped per paper. A single active task and single active DAG run avoid competing CPU inference requests. Files modified recently are deferred, and source checksums detect changes during processing. Markdown and validated JSON checkpoints use atomic file replacement. A unique filename key prevents duplicate records on retries.

Publication reads successful database commits even when another paper fails; a failure watcher preserves the overall failed DAG status. Notification status is recorded after Discord confirms delivery. An ambiguous failure between message creation and recording confirmation can still cause a duplicate. Disabled notifications remain pending. Application logs include filenames, stage transitions, elapsed times, and LLM heartbeats every 15 seconds. Airflow provides task/run state and retry history.

Local execution evidence

The scheduled run for 29 October 2026 at 17:40 UTC succeeded. It processed the complete 13-page MapReduce paper into 56,512 Markdown characters and a 13,351-token LLM prompt. Conversion took approximately 2.23 seconds; the LLM task's HTTP request took approximately 145.18 seconds with no validation retry. The full DAG run took approximately 155 seconds, including orchestration. The metadata was committed once, and the generated HTML listed the paper.

The next scheduled run, for 17:45 UTC, also succeeded. It discovered zero new papers, skipped the stored filename, and regenerated the page without calling the LLM. SQLite contained one paper and one pending notification. Discord delivery was intentionally disabled, so this is not evidence of a real delivered Discord message. Detailed run/task evidence is in docs/execution-evidence.md; progress can be followed through scripts/show_progress.py or the Airflow UI.

An additional deliberately invalid PDF failed conversion after one retry. The HTML task still succeeded, but the failure watcher correctly kept the DAG failed. The fixture was removed and a clean recovery run was triggered.

This checks that publication does not hide processing failures.

CPU feasibility, quality, and limitations

The CPU is an AMD Ryzen 9 5900HX. We used qwen2.5:1.5b with Q4_K_M quantization, eight inference threads, a 32,768-token context, and zero GPU allocation. A separate baseline benchmark took approximately 141 seconds for conversion plus first inference, with approximately 2 GiB model memory use. Repeated requests took about 10 seconds because the prompt was cached; these are not new-paper throughput measurements.

The extracted title was correct, and summaries were broadly accurate but generic. The model did not consistently follow the requested summary length. Valid JSON does not establish factual correctness, and human review against the source remains necessary. Initial scope covers text-based PDFs within the model's context budget. Scanned PDFs need OCR; arbitrary-length papers need an additional chunking strategy. Changing an already processed filename's contents is outside the assignment's append-only filename assumption.

Reproduction and final completion

The repository README documents environment installation, model download, starting Ollama and Airflow, placing PDFs, enabling the DAG, observing logs, and configuring Drive and Discord. Automated tests cover metadata validation, bounded retries, duplicate prevention, checkpoint reuse, HTML escaping, source changes, and notification recovery. Before submission, configure and verify an end-to-end Drive run and an actual Discord message, add group members, and replace this draft with the final PDF.

Reference input: Jeffrey Dean and Sanjay Ghemawat, MapReduce: Simplified Data Processing on Large Clusters, OSDI 2004. https://research.google/pubs/mapreduce-simplified-data-processing-on-large-clusters/